

Relatório de Estudo Dirigido

Otimização Combinatória para Geração de Árvores de Decisão

Aluna: Sofia Valverde Villas Bôas

RA 252974 - Engenharia de Computação AA

s252974@dac.unicamp.br

Orientador: Prof. Dr. Rafael Crivellari Saliba Schouery

1 Introdução

Suponha que um laboratório necessite estimar a eficácia de um medicamento em desenvolvimento, considerando características como idade, sexo e dosagem receitada para os indivíduos. Uma forma de abordar este problema é por meio de *árvores de decisão*, técnica utilizada para categorização de dados em classes [2]. No exemplo, cada classe está relacionada a um grau de eficácia do medicamento. A categorização ilustrada na Figura 1 ocorre via uma série de perguntas, representadas pelos retângulos, referentes às características do conjunto de dados.

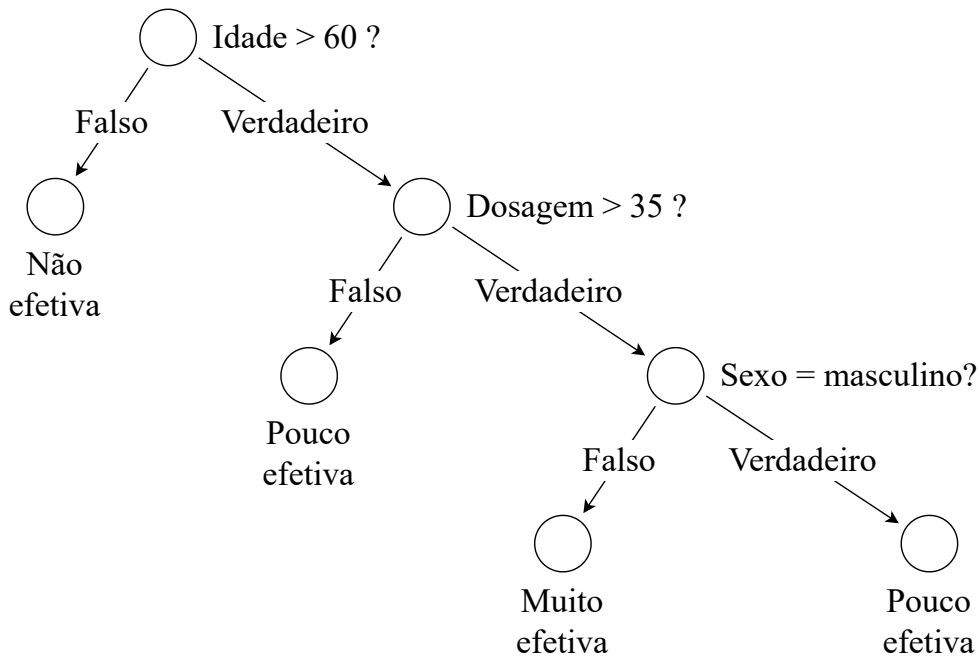


Figura 1: Exemplo de uma árvore de decisão para previsão da eficácia de um medicamento hipotético. Nós não folha da árvore representam testes aplicados às características dos dados. Os arcos correspondem ao caminho a seguir dependendo do resultado do teste. Os nós folhas são classes alvo do problema, isto é, os níveis de eficácia do medicamento e a previsão atribuída aos dados que percorrem o caminho.

Uma vantagem associada a este modelo de previsão é a facilidade de extrair informações relevantes e as decisões tomadas, conceito chamado de *explicabilidade* [2, 7, 16]. Assim, estas árvores evidenciam quais atributos foram mais importantes para a tomada de decisão. Por exemplo, o medicamento funciona eficientemente para pacientes do sexo masculino, com até 60 anos e que receberam dosagem acima de 35. Este agrupamento facilita a interpretação do modelo e a obtenção de novas hipóteses.

Algoritmos para geração de árvores de decisão tradicionalmente as constroem de forma heurística, realizando recursivamente escolhas locais ótimas em cada nó [7]. Apesar de obter resultados razoáveis na prática [14], este processo pode deixar de capturar pontos essenciais dos *datasets* [2].

Em 2017, Bertsimas e Dunn [2] propuseram a possibilidade de gerar árvores de decisão *ótimas*. Ao construir globalmente a árvore que melhor representa os dados, esta técnica

permite aliar explicabilidade e precisão na categorização.

Contudo, o problema de gerar árvores ótimas é NP-Difícil [13] e, portanto, não se conhece algoritmos que o resolva em tempo polinomial. Esta complexidade contrasta com o potencial de aplicação destes modelos na resolução de problemas práticos, como no cálculo de risco em cirurgias de emergência [3], no aumento da segurança em sistemas de energia elétrica [11] ou na recomendação de políticas sociais [5]. Apesar da dificuldade, o impacto do uso deste método evidencia a necessidade de desenvolver algoritmos computacionalmente eficientes.

Portanto, o objetivo principal deste Estudo Dirigido consiste no estudo de técnicas de Programação Linear Inteira (PLI) para a formulação do problema, em paralelo com a leitura de artigos sobre modelos e funcionamento de árvores de decisão. A partir disso, estudamos a otimalidade nestas árvores, bem como a análise do impacto de possíveis formulações e características a serem otimizadas. De forma complementar, o projeto busca introduzir a aluna às práticas acadêmicas, contribuindo para sua formação e eventual iniciação científica.

2 Atividades

O primeiro passo compreendeu o estudo dos fundamentos de Programação Linear Inteira, utilizando os livros de Wolsey [19] e Chvátal [6]. Ademais, a aluna também estudou sobre o método de geração de colunas, a partir do livro de Uchoa et al. [17]. Paralelamente, a estudante aprofundou os conhecimentos sobre árvores de decisão, de modo a conseguir perceber nuances, limitações e potencialidades desses modelos em diferentes contextos.

Em seguida, deu-se a análise de como PLI pode ser utilizada para otimizar características relevantes em árvores de decisão. Entre as referências estudadas, os trabalhos de Aghaei et al. [1], Verwer e Zhang [18] e Costa et al. [7] exploram diferentes formulações e objetivos de otimização para árvores de decisão ótimas.

Adicionalmente, o acompanhamento semanal das reuniões da equipe de pesquisa foi

importante para a troca de conhecimentos e esclarecimento de dúvidas ao longo do Estudo Dirigido. Neste contexto, a estudante realizou um seminário acerca da formulação de Verwer e Zhang [18].

Por fim, a aluna também se familiarizou com o uso de *solvers* de PLI, como o *software* Gurobi [12], ao implementar a formulação proposta por Aghaei et al. [1]. Essa etapa foi fundamental para a consolidação dos conceitos teóricos estudados ao longo do período.

3 Resultados

Em seguida, são apresentados os principais tópicos estudados ao longo do período do Estudo Dirigido.

3.1 Programação Linear Inteira

Ao decorrer do Estudo Dirigido, foram estudados diversos conceitos fundamentais de Programação Linear e Programação Linear Inteira.

Programação Linear (PL) é uma técnica utilizada no processo de modelagem e resolução de problemas de otimização [6]. Estes problemas envolvem a busca pela melhor solução, segundo um critério de custo ou valor, em um conjunto enumerável de possibilidades. Um programa linear na forma padrão é escrito como:

$$\text{maximize} \quad \sum_{j=1}^n c_j x_j \quad (1)$$

$$\text{sujeito a} \quad \sum_{j=1}^n a_{ij} x_j \leq b_i \quad i = 1, \dots, m \quad (2)$$

$$x_j \geq 0 \quad j = 1, \dots, n \quad (3)$$

A Equação 1 descreve a *função objetivo*, isto é, a função linear que será maximizada (ou minimizada) em um programa linear, no qual x_j são chamadas de *variáveis de decisão*. A Equação 2 representa as *restrições* que delimitam os possíveis valores destas variáveis.

Por fim, a Equação 3 indica uma *restrição de não negatividade*.

Denomina-se *solução viável* uma atribuição de valores a x_j que satisfaz todas as restrições impostas ao problema. Já uma *solução ótima* é uma solução viável que maximiza a função objetivo. Vale destacar que pode existir mais de uma solução ótima para o mesmo problema ou até mesmo não existir solução ótima. No caso de não existir nenhuma solução viável, o problema é chamado de *inviável* e, para o caso em que as soluções viáveis permitem que a função objetivo cresça indefinidamente, os problemas são chamados de *ilimitados* [6].

Entre as técnicas utilizadas para resolver problemas de PL, está o *método simplex* [6]. A partir de uma solução viável inicial, o *simplex* realiza iterativamente o processo de pivotamento, isto é, a troca de variáveis, visando melhorar o valor da função objetivo. Apesar de ter complexidade exponencial no pior caso, o *simplex* é rápido, na prática.

Um dos conceitos centrais em Programação Linear é a *dualidade*, isto é, a partir de um programa linear P de maximização, chamado de *primal*, é possível obter um programa D de minimização, chamado de *dual*.

$$\text{minimize} \quad \sum_{i=1}^m b_i y_i \quad (4)$$

$$\text{sujeito a} \quad \sum_{i=1}^m a_{ij} y_i \geq c_j \quad j = 1, \dots, n \quad (5)$$

$$y_i \geq 0 \quad i = 1, \dots, m \quad (6)$$

Qualquer problema linear pode ser associado a um dual, cuja função objetivo e restrições são construídas a partir dos coeficientes e parâmetros do primal.

Um *Programa Linear Inteiro* (PLI) é um PL com a restrição adicional de que todas as variáveis devem assumir valores inteiros. Determinar se um PLI tem solução viável com valor maior do que K é um problema NP-Completo. Porém, na prática, é possível resolver instâncias específicas de PLI eficientemente, graças ao desenvolvimento de algoritmos exatos, técnicas de otimização e heurísticas eficazes [19].

Uma das técnicas utilizadas para resolver problemas de PLI é o método *branch-and-bound* [19]. Iterativamente, o algoritmo resolve a *relaxação linear* do PLI, isto é, o PL obtido ao remover as restrições de integralidade, e, caso a solução não seja inteira, divide o problema original em partes menores, formando uma árvore de subproblemas. Se o valor da relaxação linear é menor que o melhor valor encontrado até o momento (*incumbente*), sabemos que a subárvore não irá produzir solução melhor. Ainda, caso a solução da relaxação linear obtida em um nó for inteira, esta solução é viável para o PLI original e pode ser utilizada para atualizar o incumbente.

3.2 Árvores de Decisão

Mediante uma série de testes referentes a características de itens de determinado conjunto de dados, árvores de decisão são capazes de classificá-los em categorias. Assim como no exemplo da Introdução, elas podem ser usadas em diversas aplicações, como na área da medicina [10, 15] ou finanças [9].

Árvores de decisão são compostas de nós internos, que estão associados a testes feitas sobre atributos dos dados, e folhas, as quais são atribuídas às categorias designadas. Elas são construídas a partir de dados em que se conhecem as categorias, processo chamado de *treinamento*. Após essa etapa, ao utilizar dados ainda não vistos, pode-se validar a previsão feita ao classificá-los corretamente.

Existem diversos algoritmos para a geração dessas árvores, sendo um dos mais influentes o algoritmo CART (Classification and Regression Trees) [4], o qual utiliza formas heurísticas para tomar decisões locais, em que são utilizadas medidas de impurezas como o *Índice de Gini*, *Entropia* ou *Razão de Ganho* [8]. Além disso, o algoritmo aplica técnicas de poda, para evitar o *overfitting* dos dados, isto é, quando o modelo se ajusta excessivamente ao conjunto de treinamento, perdendo a capacidade de generalizar para novos dados.

Algumas características são fundamentais para obter modelos mais aplicáveis ao utilizar árvores de decisão. A *explicabilidade*, mencionada na Introdução, é útil em domínios

onde as decisões precisam ser justificáveis, como medicina ou direito. No entanto, o quão explicável é um modelo está relacionado com alguns outros fatores, como o tamanho da árvore e como os cortes são feitos [7].

A *esparsidade*, se refere ao uso de um número reduzido de atributos para categorizar, o que contribui para a simplicidade do modelo e proporciona maior explicabilidade.

Ao contrário dos métodos heurísticos, como o CART [4], algoritmos para obter árvores de decisão ótimas buscam construir globalmente a árvore que melhor classifica os dados [2]. Nesse contexto, o uso de PLI surge como ferramenta para abordar o problema, possibilitando avanços no equilíbrio entre explicabilidade sem a perda de acurácia, minimização da complexidade e respeito dos critérios de justiça, garantindo soluções que são não apenas precisas, mas também interpretáveis.

3.3 Formulação para Árvores de Decisão Ótimas

Diversos trabalhos propõem formulações de PLI para a geração de árvores de decisão ótimas. Dentre estes, Aghaei et al. [1] apresentam algumas variações de modelos baseados em fluxo. O modelo inicial proposto pelos autores considera árvores balanceadas, isto é, a distância da raiz até qualquer folha (onde a classificação é feita) é a mesma e todos os nós internos possuem dois filhos.

Para uma profundidade d fixa, a árvore possui $2^{d+1} - 1$ nós, sendo que $2^d - 1$ são nós internos e pertencem ao conjunto $\mathcal{B}$, e 2^d são folhas, compondo o conjunto $\mathcal{L}$. Ademais, temos os conjuntos $\mathcal{F}$, com os atributos dos dados, $\mathcal{I}$, com as instâncias do conjunto de dados e $\mathcal{K}$, com as classes possíveis para categorização.

As variáveis utilizadas consistem em b_{nf} , que indica se o atributo f é utilizado no nó interno n ; a variável w_k^n que, indica se a folha n prevê a classe k ; e as variáveis z , que modelam o fluxo de dados através da árvore.

A seguir, apresentamos a formulação.

$$\text{maximize} \quad \sum_{i \in \mathcal{I}} \sum_{n \in \mathcal{L}} z_{n,t}^i \quad (7)$$

$$\text{sujeito a} \quad \sum_{f \in \mathcal{F}} b_{nf} = 1 \quad \forall n \in \mathcal{B}, \quad (8)$$

$$z_{a(n),n}^i = z_{n,l(n)}^i + z_{n,r(n)}^i \quad \forall n \in \mathcal{B}, i \in \mathcal{I}, \quad (9)$$

$$z_{a(n),n}^i = z_{n,t}^i \quad \forall n \in \mathcal{L}, i \in \mathcal{I}, \quad (10)$$

$$z_{s,1}^i \leq 1 \quad \forall i \in \mathcal{I}, \quad (11)$$

$$z_{n,l(n)}^i \leq \sum_{f \in \mathcal{F}: x_i^f = 0} b_{nf} \quad \forall n \in \mathcal{B}, i \in \mathcal{I}, \quad (12)$$

$$z_{n,r(n)}^i \leq \sum_{f \in \mathcal{F}: x_i^f = 1} b_{nf} \quad \forall n \in \mathcal{B}, i \in \mathcal{I}, \quad (13)$$

$$z_{n,t}^i \leq w_{y^i}^n \quad \forall n \in \mathcal{L}, i \in \mathcal{I}, \quad (14)$$

$$\sum_{k \in \mathcal{K}} w_k^n = 1 \quad \forall n \in \mathcal{L}, \quad (15)$$

$$w_k^n \in \{0, 1\} \quad \forall n \in \mathcal{L}, k \in \mathcal{K}, \quad (16)$$

$$b_{nf} \in \{0, 1\} \quad \forall n \in \mathcal{B}, f \in \mathcal{F}, \quad (17)$$

$$z_{a(n),n}^i \in \{0, 1\} \quad \forall n \in \mathcal{B} \cup \mathcal{L}, i \in \mathcal{I}, \quad (18)$$

$$z_{n,t}^i \in \{0, 1\} \quad \forall n \in \mathcal{L}, i \in \mathcal{I}. \quad (19)$$

A função objetivo (7) maximiza o número de pontos corretamente classificados. A restrição (8) garante que cada nó interno seja associado a exatamente um atributo na ramificação. As restrições (9) e (10) representam a conservação de fluxo na árvore. A restrição (11) assegura que a passagem de fluxo inicie na raiz da árvore. As restrições (12) e (13) garantem que o fluxo siga o caminho correto dependendo do valor do atributo utilizado na ramificação, e a restrição (14) assegura que uma instância só pode ser classificada em uma folha se esta for associada à classe correta. A restrição (15) garante que cada

folha seja associada a exatamente uma classe e, por fim, as restrições restantes indicam que todas as variáveis são binárias.

3.4 Experimentos

Com o objetivo de consolidar os aprendizados obtidos por meio do estudo teórico, a autora realizou uma etapa de experimentos computacionais. O modelo escolhido para implementação durante o Estudo Dirigido foi o primeiro apresentado por Aghaei et al. [1]. É válido mencionar, entretanto, que os autores propõem outras variações de modelos. Dentre estas, há a formulação para árvores desbalanceadas e a que utiliza a técnica de *Decomposição de Benders*, descrita no livro de Wolsey [19].

Todos os experimentos foram conduzidos em um processador Intel Xeon CPU E5-2630 v4 @ 2.20GHz com 64 GB de RAM. O sistema operacional utilizado foi o Ubuntu 20.04.3. Os códigos foram escritos em Python 3.10.12. Para a resolução do modelo de PLI, utilizamos o *solver* Gurobi 12.0.0 [12].

Cada instância foi executada com um limite de tempo de uma hora, semelhantemente ao realizado por Aghaei et al. [1]. Para cada conjunto de dados, analisamos o comportamento do modelo para árvores de profundidade $d \in \{2, 3, 4, 5\}$. Além disso, para garantir uma comparação justa, utilizamos somente uma *thread* durante o processo experimental.

Na Tabela 1, apresentamos as características das instâncias utilizadas.

Instância	I	F	K
soybean-small	47	45	4
monk3	122	15	2
monk1	124	15	2
hayes-roth	132	15	3
monk2	169	15	2
house-votes-84	232	16	2
spect	267	22	2
breast-cancer	277	38	2
balance-scale	625	20	3
tic-tac-toe	958	27	2
car-evaluation	1728	20	4
kr-vs-kp	3196	38	2

Tabela 1: Características dos conjuntos de dados utilizados nos experimentos. Todas as instâncias possuem apenas atributos categóricos. I representa a quantidade de dados, F o número de atributos e K o número de classes.

Na Tabela 2, reportamos o tempo de execução para resolver as instâncias testadas.

Instância	$d = 2$	$d = 3$	$d = 4$	$d = 5$
soybean-small	2.92	1.12	1.52	2.43
monk3	4.78	104.33	*	64.42
monk1	5.43	59.87	49.33	11.22
hayes-roth	4.46	33.61	*	*
monk2	12.41	1058.37	*	*
house-votes-84	4.98	140.18	987.01	245.65
spect	15.92	1153.90	*	*
breast-cancer	25.21	*	*	*
balance-scale	35.40	1142.80	*	*
tic-tac-toe	278.72	*	*	*
car-evaluation	93.91	*	*	*
kr-vs-kp	*	*	*	*

Tabela 2: Tempo de execução para resolver as instâncias testadas, em segundos. Nos casos em que o tempo limite foi atingido, indicamos por “*”.

Notamos que, ao aumentar a profundidade d da árvore para 4 ou 5, diversas instâncias não foram resolvidas no tempo limite de 1 hora. Isto pode estar relacionado ao aumento

do número de variáveis e restrições na formulação, que depende da quantidade de nós na árvore, a qual cresce exponencialmente com d .

Além disso, outro fato interessante foi que, para *monk3* e *monk1*, o tempo de execução diminui da profundidade $d = 3$ para $d = 5$. Similarmente, para *house-votes-84*, o tempo diminui de $d = 4$ para $d = 5$. Uma possível explicação para este comportamento é que, ao aumentar a profundidade, apesar do modelo crescer, a árvore pode se ajustar melhor aos dados, facilitando o processo de poda no *branch-and-bound*.

A quantidade de dados das instâncias também impactou o desempenho do modelo. Por exemplo, *soybean-small*, com 47 linhas, foi resolvida rapidamente para todas as profundidades testadas. Já *kr-vs-kp*, com 3196 linhas, não foi resolvida no tempo limite para nenhuma profundidade.

Como próximos passos, a aluna pretende continuar os experimentos e implementar as outras formulações propostas por Aghaei et al. [1], além de explorar possíveis melhorias no modelo atual. No artigo, os autores apresentam os resultados experimentais somente das formulações mais gerais baseadas em árvores desbalanceadas e as que utilizam *Decomposição de Benders*. Futuramente, pretendemos comparar de forma mais detalhada os desempenhos das diferentes formulações.

4 Conclusão

Durante o Estudo Dirigido, aspectos como uma base em PLI, disciplinas e reuniões em grupo colaboraram para o amadurecimento profissional e acadêmico da aluna. Entre eles, é importante mencionar a construção de uma base de conhecimentos em Programação Linear Inteira. Através tanto do estudo individual, como das reuniões do grupo de pesquisa semanais, a estudante consolidou o entendimento de técnicas de PLI e como elas são aplicadas no contexto de otimização.

Materiais disponibilizados de disciplinas como Fundamentos Matemáticos da Computação (MC358), Projeto e Análise de Algoritmos I (MC458) e II (MC558), e Combi-

natória Polidrica facilitaram o entendimento acerca dos conceitos estudados e abordados, por exemplo, nos livros de Chvátal [6], Wolsey [19] e Uchoa et al. [17].

Além disso, a leitura de artigos sobre árvores de decisão ótimas colaborou para a compreensão sobre o uso dessa técnica de categorização e das formulações possíveis para o problema. Entre estas, a proposta por Aghaei et al. [1], baseada em fluxo e o trabalho de Verwer e Zhang [18], o qual utiliza apenas variáveis binárias, proporcionaram à aluna um maior entendimento sobre as diferentes abordagens para o problema, bem como suas vantagens e pontos de atenção.

Por fim, na etapa de experimentação, a aluna reforçou os aprendizados adquiridos durante o Estudo Dirigido, se familiarizando com o uso do *solver* Gurobi [12] e com as boas práticas em um experimento científico.

Assinatura da Aluna

Referências

- [1] S. Aghaei, A. Gómez e P. Vayanos, “Strong Optimal Classification Trees,” *Operational Research*, 2024. DOI: 10.1287/opre.2021.0034.
- [2] D. Bertsimas e J. Dunn, “Optimal classification trees,” *Machine Learning*, v. 106, n. 7, 2017. DOI: 10.1007/s10994-017-5633-9.
- [3] D. Bertsimas, J. Dunn, G. C. Velmahos e H. M. A. Kaafarani, “Surgical Risk Is Not Linear: Derivation and Validation of a Novel, User-friendly, and Machine-learning-based Predictive OpTimal Trees in Emergency Surgery Risk (POTTER) Calculator,” *Annals of Surgery*, v. 268, n. 4, 2018. DOI: 10.1097/SLA.0000000000002956.
- [4] L. Breiman, J. H. Friedman, R. A. Olshen e C. J. Stone, *Classification And Regression Trees*. Routledge, 2017, ISBN: 9781315139470. DOI: 10.1201/9781315139470.
- [5] H. Chan, E. Rice, P. Vayanos, M. Tambe e M. Morton, “From Empirical Analysis to Public Policy: Evaluating Housing Systems for Homeless Youth,” em *Machine Learning and Knowledge Discovery in Databases*. Springer International Publishing, 2019. DOI: 10.1007/978-3-030-10997-4_5.
- [6] V. Chvátal, *Linear Programming* (Series of books in the mathematical sciences). W. H. Freeman, 1983, ISBN: 9780716715870.

- [7] V. G. Costa e C. E. Pedreira, “Recent advances in decision trees: an updated survey,” *Artificial Intelligence Review*, v. 56, n. 5, 2022. DOI: 10.1007/s10462-022-10275-5.
- [8] E. Demirović et al., “MurTree: Optimal Classification Trees via Dynamic Programming and Search,” 2020. DOI: 10.48550/ARXIV.2007.12652.
- [9] V. Dhar, D. Chou e F. Provost, “Discovering Interesting Patterns for Investment Decision Making with GLOWER—A Genetic Learner Overlaid with Entropy Reduction,” *Data Mining and Knowledge Discovery*, v. 4, n. 4, 2000. DOI: 10.1023/a:1009848126475.
- [10] A. A. Freitas, “Comprehensible classification models: a position paper,” *ACM SIGKDD Explorations Newsletter*, v. 15, n. 1, 2014. DOI: 10.1145/2594473.2594475.
- [11] I. Genc, R. Diao, V. Vittal, S. Kolluri e S. Mandal, “Decision Tree-Based Preventive and Corrective Control Applications for Dynamic Security Enhancement in Power Systems,” *IEEE Transactions on Power Systems*, v. 25, n. 3, 2010. DOI: 10.1109/tpwrs.2009.2037006.
- [12] “Gurobi Optimizer,” acesso em 26 de out. de 2025. endereço: <https://www.gurobi.com>.
- [13] L. Hyafil e R. L. Rivest, “Constructing optimal binary decision trees is NP-complete,” *Information Processing Letters*, v. 5, n. 1, 1976. DOI: 10.1016/0020-0190(76)90095-8.
- [14] J. G. M. van der Linden, D. Vos, M. M. de Weerdt, S. Verwer e E. Demirović, “Optimal or Greedy Decision Trees? Revisiting their Objectives, Tuning, and Performance,” 2024. DOI: 10.48550/ARXIV.2409.12788.
- [15] G. Richards, V. Rayward-Smith, P. Sönksen, S. Carey e C. Weng, “Data mining for indicators of early mortality in a database of clinical records,” *Artificial Intelligence in Medicine*, v. 22, n. 3, 2001. DOI: 10.1016/s0933-3657(00)00110-x.
- [16] C. Rudin, “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead,” *Nature Machine Intelligence*, v. 1, n. 5, 2019. DOI: 10.1038/s42256-019-0048-x.
- [17] E. Uchoa, A. Pessoa e L. Moreno, “Optimizing with Column Generation: Advanced Branch-Cut-and-Price Algorithms (Part I),” *Cadernos do LOGIS-UFF*, Universidade Federal Fluminense, Engenharia de Produção, rel. técn. L-2024-3, 2024.
- [18] S. Verwer e Y. Zhang, “Learning Optimal Classification Trees Using a Binary Linear Program Formulation,” *Proceedings of the AAAI Conference on Artificial Intelligence*, v. 33, n. 01, 2019. DOI: 10.1609/aaai.v33i01.33011624.

- [19] L. A. Wolsey, *Integer programming*, Second edition. Hoboken, NJ: Wiley, 2021, 1316 pp., ISBN: 9781119606529.